Guía 05 · Filtrado, notch y remuestreo

Oct 15, 2026 · @Daniel Gómez

Objetivo y orden del preprocesado

Aquí limpias la señal de lo que no es cerebro (deriva lenta, red eléctrica, ruido rápido) y **unificas la frecuencia de muestreo**, imprescindible en TUSZ porque conviven 250, 256, 400, 512 y 1 000 Hz. El resultado es `preprocess()`, la función que aplicarás a cada fichero del TFM.

Qué	Valor
Notebook	`Curso_MNE/05_Filtrado_y_remuestreo.ipynb`
Fase del TFM	Fase 2 · Preprocesado
Duración	~1 h
Función final	`preprocess(raw, l_freq=0.5, h_freq=40, notch=60, sfreq=200, tcp=True)`

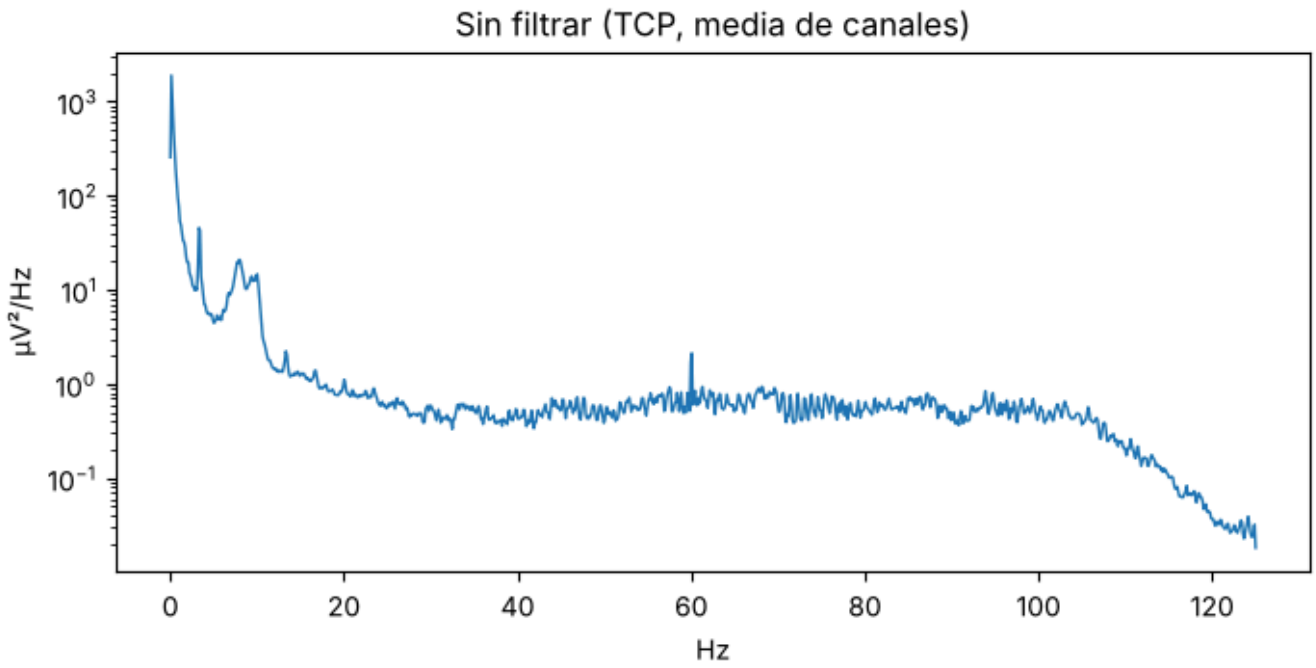
El orden importa. Esto es lo que hace `preprocess`, paso a paso:

1. Leer el EDF y limpiar nombres y tipos (`read_tusz_raw`, Guía 01).
2. Pasar a montaje TCP de 22 canales (`make_tcp`, Guía 04).
3. Notch a 60 Hz para quitar la red eléctrica.
4. Paso banda 0,5-40 Hz.
5. Remuestrear a 200 Hz.
6. Guardar en FIF para no repetirlo nunca.

Regla de oro de este módulo: se filtra la **señal continua** antes de cortar ventanas, nunca cada ventana por separado. Verás por qué en el apartado del filtro FIR.

Punto de partida y notch a 60 Hz

Antes de filtrar, mira siempre el espectro de la señal tal cual (aquí ya en TCP, media de los 22 canales).

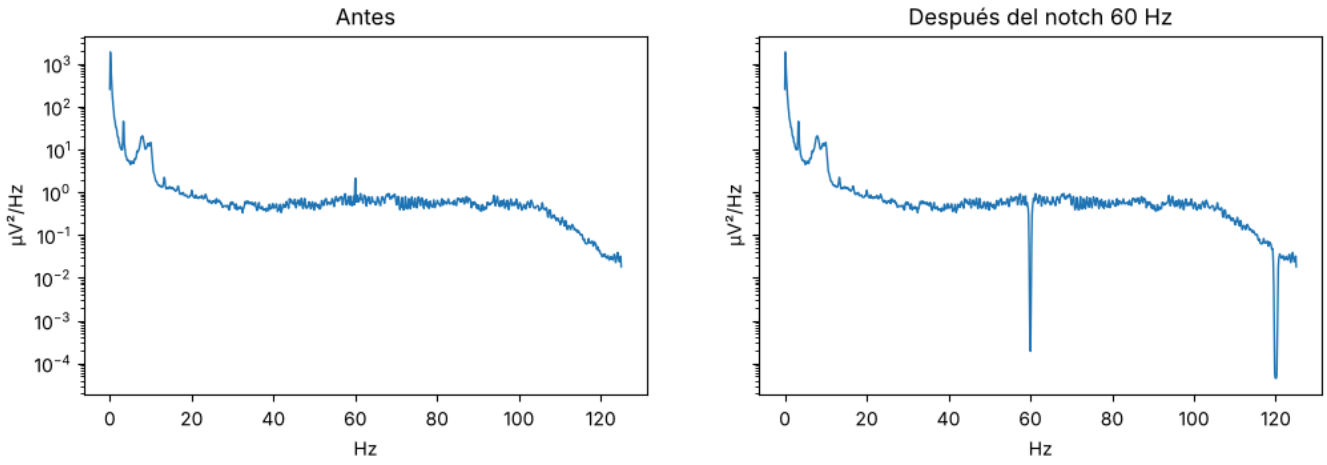


Cómo leerla. Dos problemas a la vista:

- **Potencia enorme por debajo de 0,5 Hz** (el pico pegado a 0 Hz): deriva lenta por sudor, movimiento o electrodos. No es actividad cerebral útil.
- **Pico estrecho a 60 Hz**: la red eléctrica de EE. UU. Su armónico a 120 Hz queda junto a Nyquist (125 Hz), donde el filtro del propio equipo hace caer la curva.

El filtro notch elimina una frecuencia muy concreta y deja el resto intacto.

```
raw_n = raw.copy().notch_filter(freqs=[60, 120])   # 120 solo si sfreq > 240
```



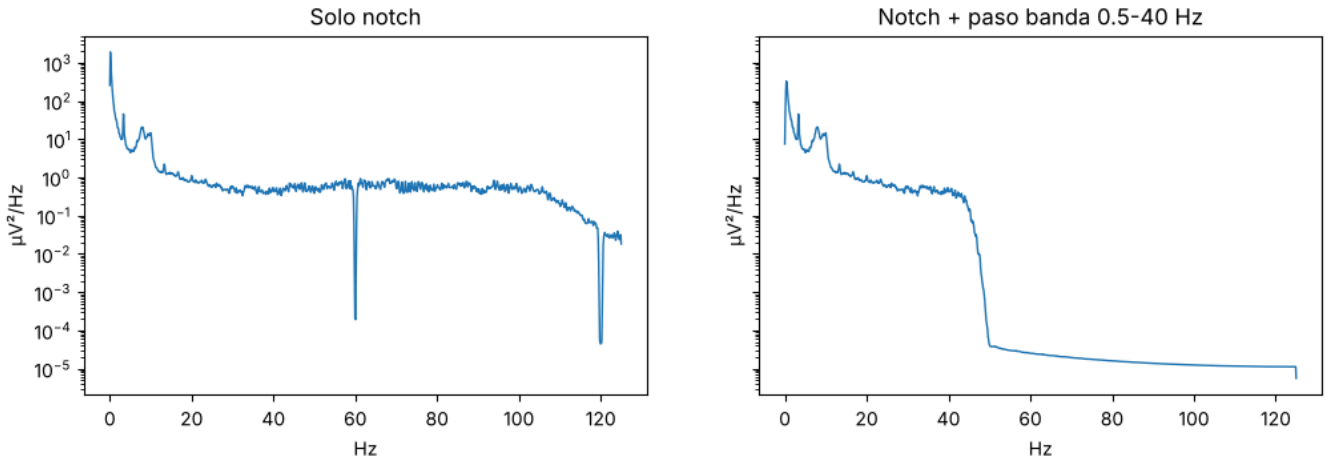
Cómo leerla. A la derecha, el pico de 60 Hz se convierte en un **valle estrecho** (y otro en 120 Hz). Todo lo demás queda igual. Eso es justo lo que debe hacer un notch.

¿Hace falta si luego filtro a 40 Hz? Casi no: el paso bajo ya elimina los 60 Hz. Aun así se suele aplicar porque no cuesta nada y evita que una línea tan potente cree *ringing* (oscilaciones espurias) en el paso bajo.

Paso banda 0,5-40 Hz

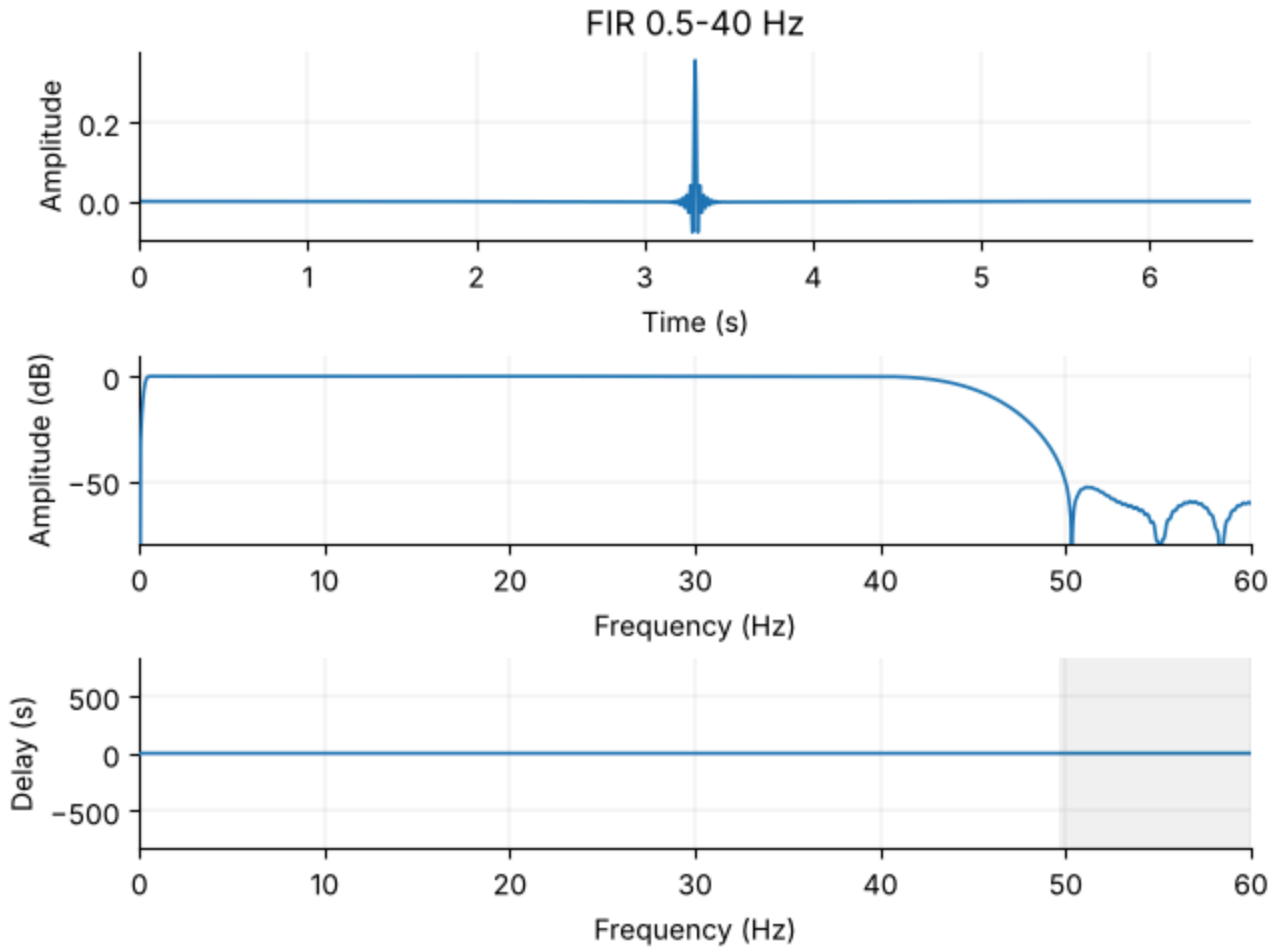
`raw.filter(l_freq, h_freq)` combina dos filtros:

- **`l_freq` = paso alto:** quita lo más lento. Típico en crisis: 0,5 Hz (o 1 Hz).
- **`h_freq` = paso bajo:** quita lo más rápido. Típico: 40-50 Hz. Algunos trabajos suben a 70-100 Hz para captar actividad rápida al inicio de crisis focales.
- Por defecto MNE usa un **FIR de fase cero**: no desplaza la señal en el tiempo, así que las anotaciones siguen cuadrando.



Cómo leerla. A la derecha, por encima de ~45 Hz la potencia cae en picado: todo lo que no te interesa desaparece. Por debajo de 40 Hz la curva es idéntica.

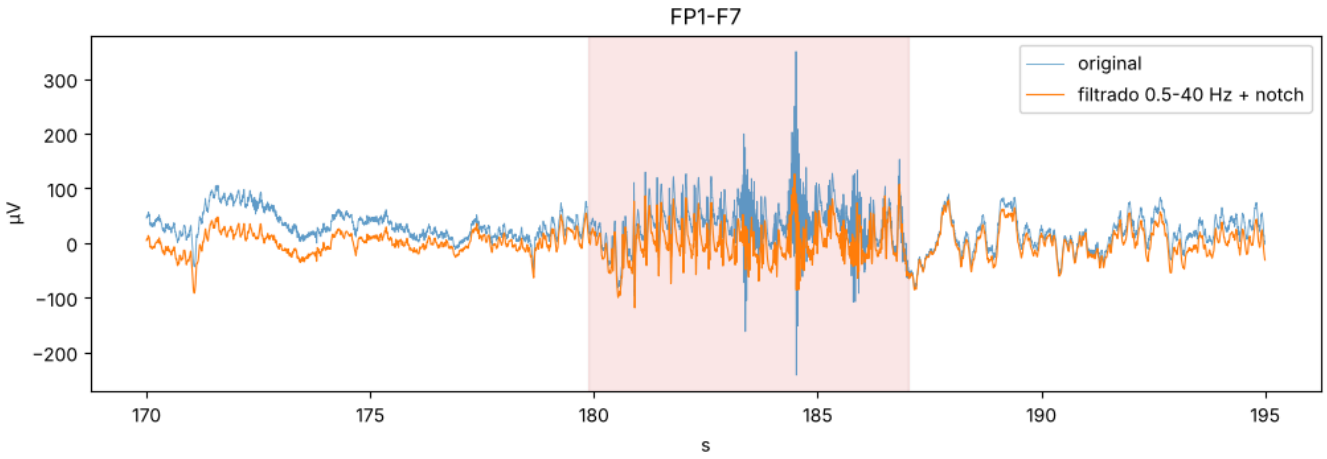
El filtro por dentro. `mne.filter.create_filter` + `mne.viz.plot_filter` muestran cómo es el filtro que vas a aplicar. Es la figura para justificar tus elecciones en la memoria.



Cómo leerla. Tres paneles:

- **Arriba (tiempo):** el filtro dura **1 651 muestras = 6,6 s**. Un paso alto a 0,5 Hz necesita mucho contexto para distinguir una onda lenta de una deriva.
- **Centro (amplitud):** 0 dB (pasa todo) entre 0,5 y 40 Hz y una caída fuerte a partir de ~45 Hz.
- **Abajo (retardo):** 0 s en todas las frecuencias. Fase cero: no mueve nada en el tiempo.

Por eso se filtra la señal continua. Si filtraras cada ventana de 4 s por separado, el filtro de 6,6 s no cabría: tendrías artefactos en los bordes de cada ventana.



Cómo leerla. La señal filtrada sigue la original pero sin la deriva lenta (ya no sube y baja lentamente) y algo más suave. La punta-onda de la crisis se conserva entera: el filtro no ha borrado lo que te interesa.

Alternativa IIR (Butterworth). `raw.filter(0.5, 40, method="iir", iir_params=dict(order=4, ftype="butter"))`. Es más corto y barato; MNE lo aplica hacia delante y hacia atrás (`filtfilt`) para que también sea de fase cero.

Remuestreo y la función `preprocess`

Por qué remuestrear. Una red neuronal espera siempre el mismo número de muestras por ventana. En TUSZ no es así:

Frecuencia de muestreo	Nº de ficheros en TUSZ
256 Hz	6 071
250 Hz	1 171
400 Hz	658
512 Hz	130
1 000 Hz	110

Una ventana de 4 s tendría 1 000, 1 024, 1 600, 2 048 o 4 000 muestras según el fichero. La solución es llevarlo todo a una frecuencia común.

Por qué 200 Hz. Su Nyquist (100 Hz) deja mucho margen sobre el paso bajo de 40 Hz y ocupa un 20 % menos de memoria que 250 Hz. Otras elecciones habituales: 128 o 256 Hz.

```
raw_200 = raw_f.copy().resample(200)
# 250 Hz → 200 Hz | 359 250 → 287 400 muestras | las 3 anotaciones absz se
conservan
```

`resample` aplica su propio anti-*aliasing*, pero es buena práctica filtrar paso bajo **antes**. Las anotaciones están en segundos, así que no se ven afectadas.

La función del curso.

```
def preprocess(raw, l_freq=0.5, h_freq=40.0, notch=60.0, sfreq=200.0,
tcp=True):
    out = make_tcp(raw) if tcp else raw.copy().pick("eeg")
    if notch and notch < out.info["sfreq"] / 2:
        out.notch_filter(notch)
    out.filter(l_freq, h_freq)
    if sfreq and out.info["sfreq"] != sfreq:
        out.resample(sfreq)
    return out

clean = preprocess(read_tusz_raw("aaaaabqv_s001_t000"))   # 22 × 287 400, 200
Hz
clean.save("salidas/aaaaabqv_s001_t000_prep_raw.fif")
```

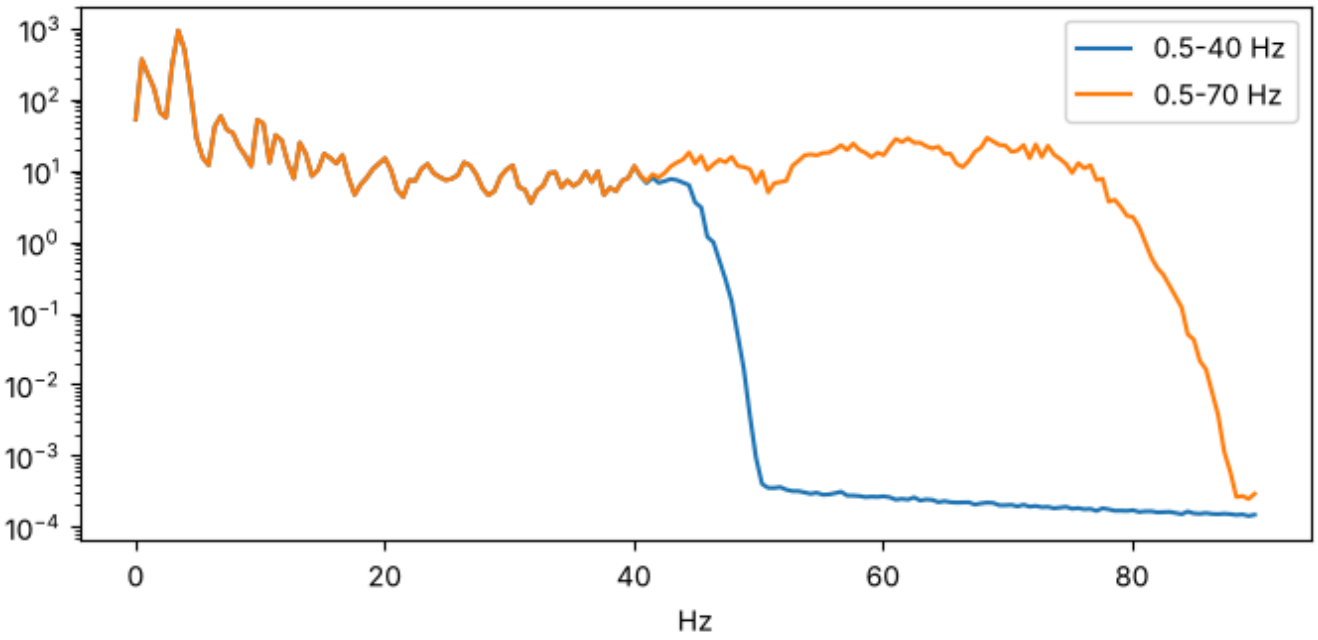
Preprocesa una vez y guarda en FIF. Con los 81 GB de TUSZ, repetir el preprocesado en cada experimento es tiempo perdido.

Cuándo usar cada filtro

Problema	Herramienta	Valores típicos	Ojo con
Red eléctrica	`notch_filter(60)`	60 Hz (+120 si cabe)	En Europa sería 50 Hz; TUSZ es de EE. UU.
Deriva lenta	Paso alto `filter(l_freq, None)`	0,5-1 Hz	Cuanto más bajo, más largo el filtro
Ruido rápido / músculo	Paso bajo `filter(None, h_freq)`	40-50 Hz (70-100 para focales)	Puedes perder actividad rápida ictal
Ambos	`filter(0.5, 40)` (FIR fase cero)	—	Siempre sobre la señal continua

Problema	Herramienta	Valores típicos	Ojo con
Filtro más barato	`method="iir"` Butterworth	orden 4	MNE lo hace de fase cero
Distintas `sfreq`	`resample(200)`	128, 200 o 256 Hz	Filtra paso bajo antes
Todo junto	`preprocess(raw)`	0,5-40 Hz, notch 60, 200 Hz	Guarda en FIF después
Justificar el filtro en la memoria	`create_filter` + `plot_filter`	—	Muestra longitud y fase

Ejercicios y resultados



E5.1 · ¿40 o 70 Hz? Las dos curvas son iguales hasta 40 Hz; a partir de ahí la de 40 Hz cae. En una ausencia casi toda la energía está por debajo de 20 Hz, así que **40 Hz no pierde nada importante**. Para crisis focales con actividad rápida conviene probar 70 Hz.

E5.2 · Mismo formato para un fichero AR. `preprocess` de `aaaaamtc_s001_t000` da 22 canales a 200 Hz, igual que el LE. Los dos ya son intercambiables para la red.

E5.3 · ¿Cuánto tarda? 3,6 s por fichero de 24 min en un núcleo, lo que da **~3,6 h para todo TUSZ** (~1 475 h de registro). Con `joblib` y varios núcleos baja a menos de una hora. (Medido en el entorno donde se ejecutaron los notebooks; en tu portátil puede variar.)

Autoevaluación

- [ ] ¿Por qué no se filtra cada ventana de 4 s por separado? → El FIR dura 6,6 s: bordes con artefactos.
- [ ] ¿Qué significa “fase cero”? → El filtro no desplaza la señal en el tiempo.
- [ ] ¿Por qué 60 Hz y no 50 Hz? → TUSZ se registró en EE. UU.
- [ ] ¿Por qué remuestrear? → Misma longitud de ventana para todos los ficheros.
- [ ] ¿Qué orden sigue `preprocess`? → TCP → notch → paso banda → remuestreo.

Siguiente: Guía 06 · Artefactos e ICA.